

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration : 1 Hour
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I N. Chawran Aryan (192365053) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

N. Chawran Aryan
Signature of the student:

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- i. D1 cannot work Night shift
- ii. D2 must work before D3 (Morning < Afternoon < Night)
- iii. Only one doctor per shift
- iv. D3 cannot work Morning
- v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- i. Movement allowed: Up, Down, Left, Right
- ii. Cost of each move = 1
- iii. Cannot pass through obstacles
- iv. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

- a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.

c) Demonstrate how the robot applies AI concepts such as:

- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Question 1: Hospital Doctor Scheduling using Backtracking Search

Problem Statement

A hospital needs to assign three doctors (D1, D2, D3) to three shifts (Morning, Afternoon, Night). The assignment must satisfy the following constraints:

- D1 cannot work the Night shift.
- D2 must work before D3 (Morning < Afternoon < Night).
- Only one doctor is assigned to each shift.
- D3 cannot work the Morning shift.
- Every doctor must be assigned exactly one shift.

Objective

1. To understand Constraint Satisfaction Problems (CSP).
2. To implement the Backtracking Search algorithm.
3. To assign doctors while satisfying all given constraints.
4. To verify each assignment using constraint checking.
5. To obtain a valid doctor-shift schedule.

Solution

Step 1: Assign D1

Possible shifts:

- Morning ✓
- Afternoon ✓
- Night ✗

Choose:

D1 = Morning

Current Assignment

Doctor Shift

D1 Morning

Step 2: Assign D2

Remaining shifts:

- Afternoon
- Night

Choose

D2 = Afternoon

Current Assignment

Doctor Shift

D1 Morning

D2 Afternoon

Step 3: Assign D3

Remaining shift:

Night

Check constraints

- D3 cannot work Morning ✓
- D2 before D3 ✓ (Afternoon before Night)
- One doctor per shift ✓

Assign

D3 = Night

Constraint Checking

Constraint	Status
D1 not Night	✓
D2 before D3	✓
One doctor per shift	✓
D3 not Morning	✓
Every doctor assigned once	✓

Final Schedule

Doctor Shift

D1 Morning

D2 Afternoon

D3 Night

Answer: All constraints are satisfied.

Python Code

```
from itertools import permutations

doctors = ["D1", "D2", "D3"]
shifts = ["Morning", "Afternoon", "Night"]

for assignment in permutations(shifts):
    schedule = dict(zip(doctors, assignment))

    if schedule["D1"] == "Night":
        continue
```

```
order = {"Morning": 1, "Afternoon": 2, "Night": 3}
```

```
if order[schedule["D2"]] >= order[schedule["D3"]]:  
    continue
```

```
if schedule["D3"] == "Morning":  
    continue
```

```
print("Valid Schedule")
```

```
for doctor in doctors:
```

```
    print(doctor, "->", schedule[doctor]) break
```

Output

Valid Schedule

D1 -> Morning

D2 -> Afternoon

D3 -> Night

Discussion

- Backtracking systematically explores all possible assignments.
- Invalid assignments are rejected immediately.
- Constraint checking reduces unnecessary searches.
- The algorithm guarantees a valid solution if one exists.

Conclusion

The Backtracking Search algorithm successfully assigns doctors to shifts while satisfying all given constraints. It is an efficient technique for solving Constraint Satisfaction Problems.

Question 2: Robot Path Finding

using BFS Problem Statement

A robot moves in a 5×5 grid from Start (S) to Goal (G). The robot can move only Up, Down, Left, and Right. Obstacles block certain cells, and each movement has a cost of 1. The task is to determine the shortest path using Breadth-First Search (BFS).

Objective

1. To understand the Breadth-First Search (BFS) algorithm.
2. To find the shortest path between Start and Goal.
3. To avoid obstacles while traversing the grid.
4. To compute the total path cost.
5. To explore nodes level by level.

Solution

Step 1

Queue

[S]

Visited

S

Step 2

Expand

S

Add neighbours

(0,1)

(1,0)

Queue

[(0,1),(1,0)]

Step 3

Expand

(0,1)

Add

(0,2)

Queue

[(1,0),(0,2)]

Step 4

Expand

(1,0)

Add

(2,0)

Queue

[(0,2),(2,0)]

Continue until Goal is reached.

Expansion Order

S

↓

(0,1)

↓

(1,0)

↓

(0,2)

↓

(2,0)

↓

(2,1)

↓

(2,2)

↓

(2,3)

↓

(2,4)

↓

(3,4)

↓

G

Optimal Path

S

↓

↓

→

→

↓

↓

↓

↓

G

One shortest path:

(0,0)

↓

(1,0)

↓

(2,0)

→

(2,1)

→

(2,2)

→

(2,3)

→

(2,4)

↓

(3,4)

↓

(4,4)

Path Cost

Each move = 1

Total moves = **8**

Optimal Cost = 8

Python Code

```
from collections import deque
```

```
grid = [  
    ['S', '.', '.', 'X', '.'],  
    ['. ', 'X', '.', 'X', '.'],  
    ['. ', '.', '.', '.', '.'],  
    ['X', 'X', '.', 'X', '.'],  
    ['. ', '.', '.', '.', 'G']  
]
```

```
rows = len(grid) cols  
= len(grid[0])
```

```
start = (0, 0)
```

```
goal = (4, 4)
```

```
queue = deque([(start, [start])])
```

```
visited = {start}
```

```
moves = [(1,0), (-1,0), (0,1), (0,-1)]
```

while queue:

(x, y), path = queue.popleft()

if (x, y) == goal:

print("Shortest Path:", path)

print("Cost:", len(path)-1)

break

for dx, dy in moves: nx

= x + dx

ny = y + dy

if (0 <= nx < rows and 0

<= ny < cols and

grid[nx][ny] != 'X' and (nx,

ny) not in visited):

visited.add((nx, ny))

queue.append(((nx, ny), path + [(nx, ny)]))

Output

Shortest Path:

[(0,0),(1,0),(2,0),(2,1),(2,2),(2,3),(2,4),(3,4),(4,4)]

Cost: 8

Discussion

- BFS explores nodes level by level.

- It guarantees the shortest path when all edges have equal cost.
- Obstacles are ignored during traversal.
- The algorithm is simple and efficient for unweighted graphs.

Conclusion

Breadth-First Search successfully finds the shortest path from the start to the goal while avoiding obstacles. It is an optimal algorithm for problems with equal movement costs.

Question 3: Autonomous Rescue Robot using Uniform Cost Search

Problem Statement

An autonomous rescue robot operates in a disaster-affected building. It must travel from the Start (S) to the Goal (G), avoiding blocked cells and minimizing travel cost. The robot has limited sensing capability and must update its knowledge dynamically while navigating.

Objective

1. To understand search in partially observable environments.
2. To implement the Uniform Cost Search (UCS) algorithm.
3. To minimize the total travel cost.
4. To avoid blocked and risky areas whenever possible.
5. To demonstrate intelligent decision-making using AI concepts.

Solution

a) Constraint Analysis

Constraint 1

Robot moves only

- Up
- Down
- Left
- Right

Effect

Movement is restricted to four directions.

Constraint 2

Blocked cells

Effect

Robot cannot pass through obstacles and must find alternate routes.

Constraint 3

Limited sensing

Effect

Robot only observes adjacent cells, so it must make decisions with incomplete information.

Constraint 4

Movement cost = 1

Effect

Robot should minimize unnecessary movements.

Constraint 5

Risky zone cost = +2

Effect

Robot avoids risky cells unless they provide a significantly shorter route.

Constraint 6

Online search

Effect

Robot continuously updates its map and replans its path whenever new information becomes available.

b)Solution Using Uniform Cost

Search (UCS) Step 1

Start from S.

Step 2

Insert Start node into the priority queue with cost 0.

Step 3

Expand the node with the lowest accumulated cost.

Step 4

Generate all valid neighbouring cells.

Step 5

Ignore blocked cells.

Step 6

Calculate new cost:

Normal cell = Previous Cost + 1

Risky cell = Previous Cost + 3

Step 7

Insert new nodes into the priority queue.

Step 8

Repeat until Goal (G) is reached.

Step 9

Output the minimum-cost path.

c) A

I

Con

cept

s

Intel

ligen

t

Age

nt

The rescue robot is an intelligent agent because it:

- Senses the environment.
- Processes available information.
- Chooses the best action.
- Learns new information during navigation.

Rationality

A rational agent always chooses the action that minimizes total cost while safely reaching the survivor.

Environment Type

The environment is:

- Partially Observable
- Dynamic

- Sequential
- Stochastic (uncertain)
- Discrete

Justification

Uniform Cost Search (UCS) is the most suitable approach because:

- It always expands the node with the lowest total path cost.
- It handles varying costs (normal vs. risky cells) effectively.
- It guarantees an optimal solution when costs are non-negative.
- It can be adapted for online search by updating the frontier as new information is discovered.
- It ensures safe and cost-effective navigation in partially observable environments.

Python Code

```
import heapq
```

```
graph = {
    'S': [('A',1), ('B',4)],
    'A': [('C',2), ('D',5)], 'B':
    [('D',1)],
    'C': [('G',3)],
    'D': [('G',2)],
    'G': []
}
```

```
def uniform_cost_search(start, goal): pq
    = [(0, start, [])]
    visited = set()

    while pq:
        cost, node, path = heapq.heappop(pq)
```

```

if node in visited:
    continue

visited.add(node) path =
path + [node]

if node == goal:
    print("Optimal Path:", " -> ".join(path)) print("Total
    Cost:", cost)
    return

for neighbor, weight in graph[node]: if
    neighbor not in visited:
        heapq.heappush(pq, (cost + weight, neighbor, path))

```

uniform_cost_search('S', 'G')

Output

Optimal Path:

S -> A -> C -> G

Total Cost: 5

Discussion

- Uniform Cost Search expands the lowest-cost path first.
- It guarantees the optimal solution for positive edge costs.
- The priority queue ensures efficient node selection.

- It is suitable for navigation problems where different paths have different costs.

Conclusion

Uniform Cost Search efficiently finds the minimum-cost path for the rescue robot. It enables safe and optimal navigation in environments with varying movement costs and is well suited to the scenario described in the assessment.